Code & Test Coverage

Do your tests cover most of the possibilities?

Óscar Barrios

Test Automation Specialist @SUSE

1. Test coverage vs Code coverage
2. Why TDD?
3. Write a test for your spec
4. Write a method for your test
5. Check Cyclomatic complexity
6. Improve your code
7. Check Code coverage
8. Improve your test

1 Test coverage vs Code coverage

Code coverage is a **metric** to know how much code (and what code) of the application is executed during the execution of our tests.

- Branch Coverage
- Function Coverage
- Statement Coverage
- Loop Coverage
- Condition Coverage
- Finite State Machine Coverage

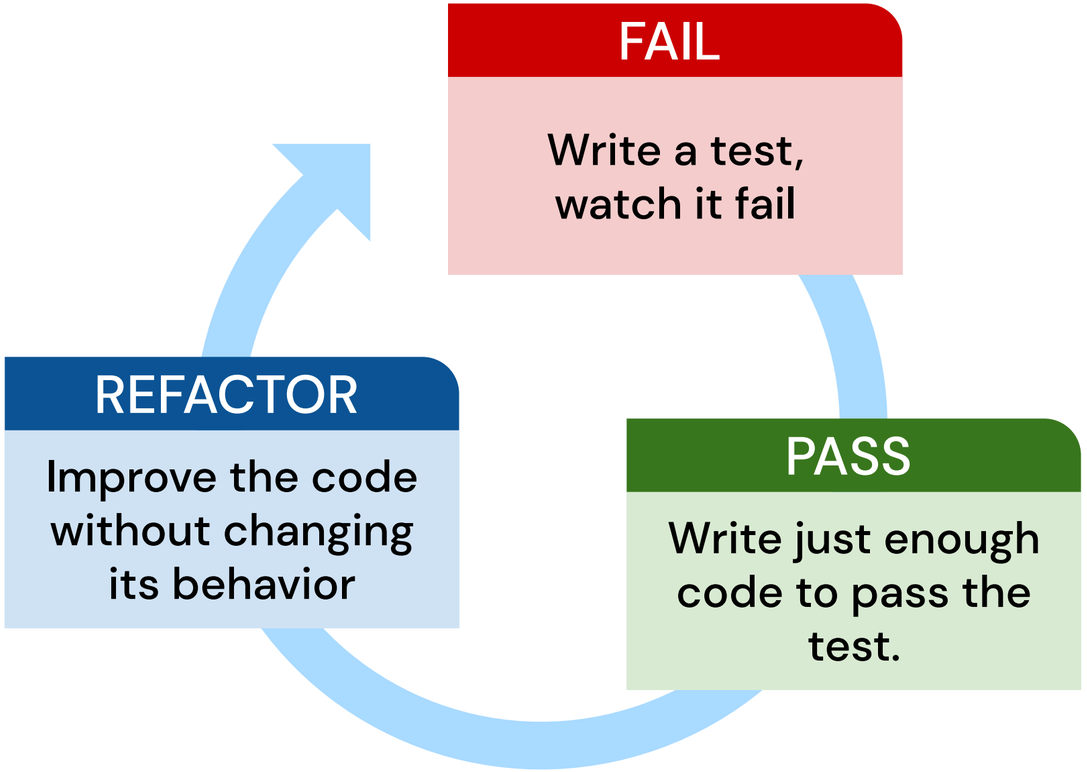
Test coverage is a **subjective estimation** to identify which areas of the application are tested, how many requirements are covered and risks evaluated.

- Features Coverage
- Risk Coverage
- Requirements Coverage

Why TDD?

In Test-Driven Development **the requirements are translated into tests**, in this way, when the tests pass, it will be guaranteed that the software meets the requirements that have been established.

If we write the tests first and the implemented code makes the test pass, it will be **easier to cover all possible paths**.



3 Write a test for your spec

- Given a chatbot that has a method where we pass a message and the age of an user
- I want that:
  - if I tell it “Hi” it answers “Hello”
  - if I tell it “Bye” it replies “See you!”
  - if I a user is older than 18 and I tell a random message, then it replies with a random message
    - otherwise it replies with “I’m not allowed to chat with you, sorry.”

3 Write a test for your spec

- Add an empty method
  - receives a String
  - returns null over a String type
- Test to send the text “Hi” to the method
- Test to send the text “Bye” to the method
- Test to send a random text to the method
- All the tests should be red at the beginning

- Refactor your method to make these tests pass

```java
package codecovarage;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

class ChatBotFirstTest {

    FirstChatBot bot;

    @BeforeEach
    void setUp() {
        bot = new FirstChatBot(List.of("alpha", "bravo", "charlie", "delta", "echo"));
    }

    @Test
    void testHi() {
        String answer = bot.chat("Hi");
        assertEquals( expected: "Hello", answer);
    }

    @Test
    void testBye() {
        String answer = bot.chat("Bye");
        assertEquals( expected: "See you!", answer);
    }

    @Test
    void testAnotherMessage() {
        String answer = bot.chat("Good morning");
        assertNotNull(answer);
    }
}
```

4 Write a method for your test

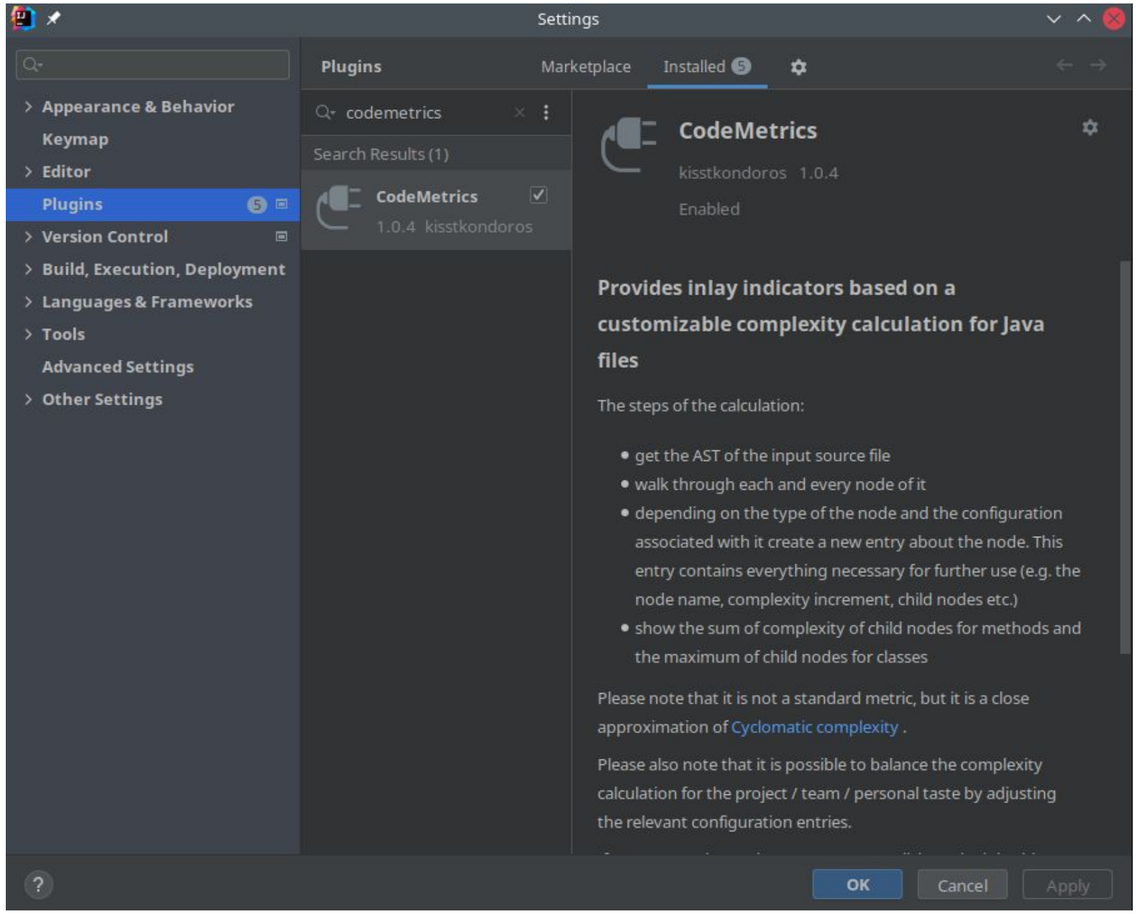
- Using TDD, write more test cases and refactor your code, so every new test case included can pass.

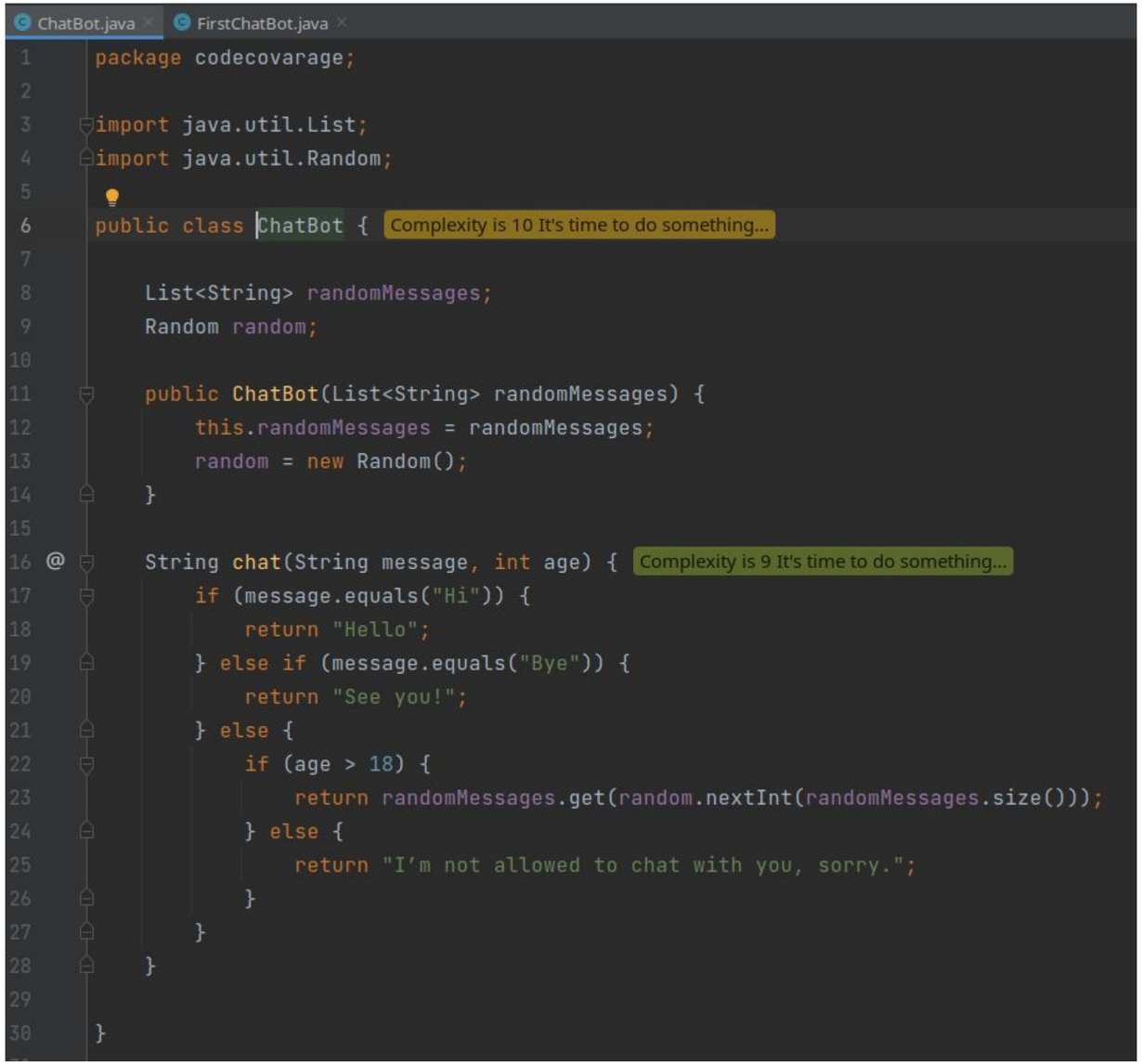
- When you finish, you should have a set of unit tests covering all the possibilities described in the specifications and a method that implements all the code need to satisfy those tests.

```java
class ChatBotSecondTest {

    ChatBot bot;
    List<String> botMessages = List.of("alpha", "bravo", "charlie", "delta", "echo");

    @BeforeEach
    void setUp() { bot = new ChatBot(botMessages); }

    @Test
    @Tag("younger")
    void testHiYoungerThanEighteen() {
        String answer = bot.chat( message: "Hi",  age: 17);
        System.out.println(answer);
        assertEquals( expected: "Hello", answer);
    }

    @Test
    void testHiOlderThanEighteen() {
        String answer = bot.chat( message: "Hi",  age: 19);
        System.out.println(answer);
        assertEquals( expected: "Hello", answer);
    }

    @Test
    @Tag("younger")
    void testByeYoungerThanEighteen() {
        String answer = bot.chat( message: "Bye",  age: 17);
        System.out.println(answer);
        assertEquals( expected: "See you!", answer);
    }
```

```java
    @Test
    void testByeOlderThanEighteen() {
        String answer = bot.chat( message: "Bye",  age: 19);
        System.out.println(answer);
        assertEquals( expected: "See you!", answer);
    }

    @Test
    @Tag("younger")
    void testAnotherMessageYoungerThanEighteen() {
        String answer = bot.chat( message: "Good morning",  age: 17);
        System.out.println(answer);
        assertEquals( expected: "I'm not allowed to chat with you, sorry.", answer);
    }

    @Test
    void testAnotherMessageOlderThanEighteen() {
        String answer = bot.chat( message: "Good morning",  age: 19);
        System.out.println(answer);
        assertTrue(botMessages.contains(answer));
    }
}
```

5 Check Cyclomatic complexity





6 Improve your code

All in codecoverage-workshop

Project

- codecoverage-workshop ~/IdeaProjects/codec
  - .idea
  - src
    - main
      - java 100% classes, 100% lines covered
        - codecovarage 100% classes, 100% l
          - ChatBot 100% methods, 100% lir
          - FirstChatBot 100% methods, 100
          - RefactoredChatBot 100% metho
    - test
      - java
        - codecovarage
          - ChatBotFirstTest
          - ChatBotSecondTest
          - ChatBotThirdTest
  - target
  - pom.xml
- External Libraries
- Scratches and Consoles

RefactoredChatBot.java | ChatBot.java | ChatBotSecondTest.java | ChatBotThirdTest.java

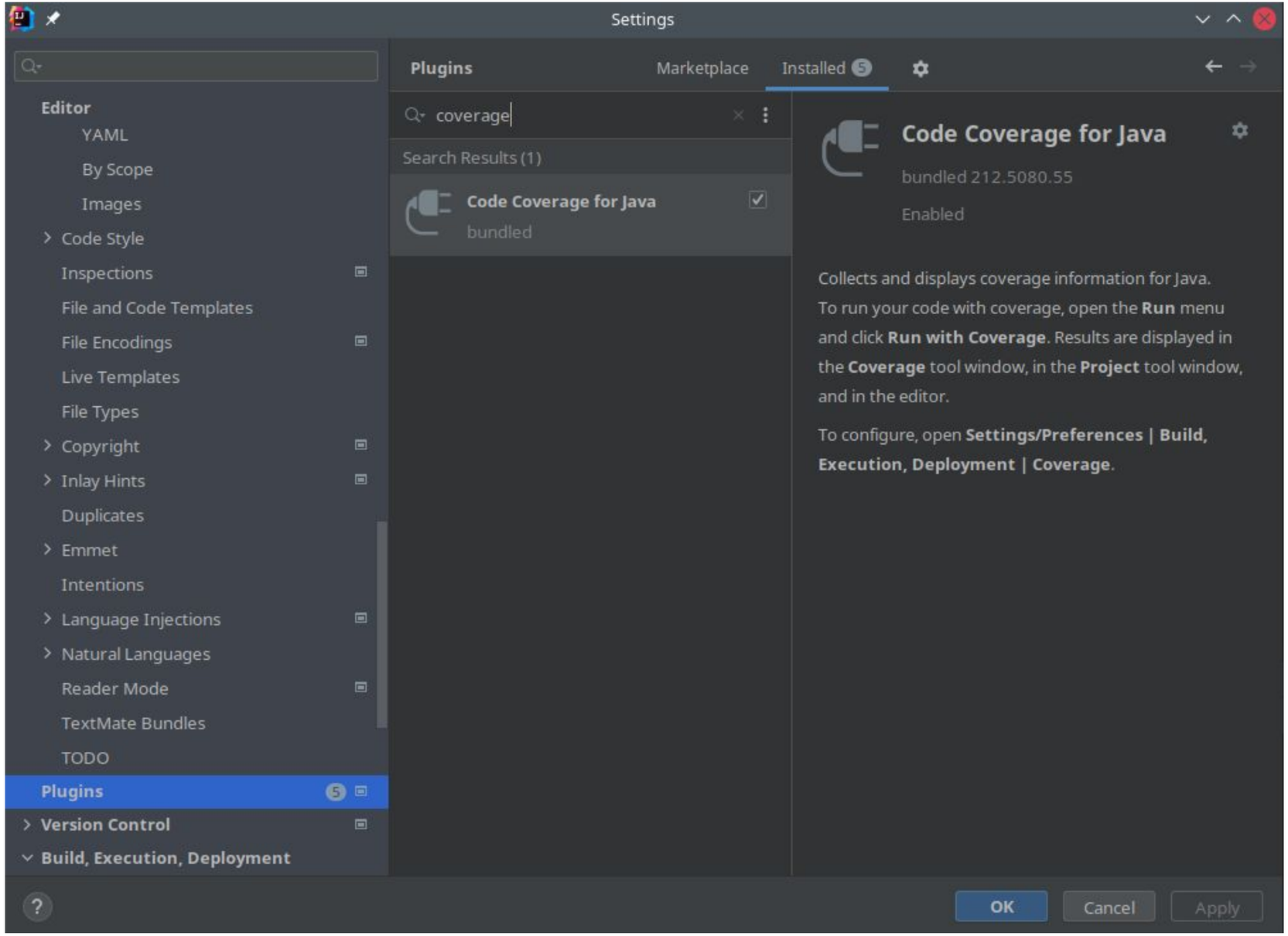
```java
package codecovarage;

import java.util.List;
import java.util.Random;

public class RefactoredChatBot {  Complexity is 6 It's time to do something...

    List<String> randomMessages;
    Random random;

    public RefactoredChatBot(List<String> randomMessages) {
        this.randomMessages = randomMessages;
        random = new Random();
    }

    String chat(String message, int age) {  Complexity is 4 Everything is cool!
        if (isGreeting(message)){
            return greetings(message);
        }
        return randomMessage(age);
    }

    private String randomMessage(int age) {  Complexity is 5 Everything is cool!
        if (age > 18) {
            return randomMessages.get(random.nextInt(randomMessages.size()));
        } else {
            return "I'm not allowed to chat with you, sorry.";
        }
    }

    private String greetings(String message) {  Complexity is 4 Everything is cool!
        if (message.contains("Hi")){
            return "Hello";
        } else {
            return "See you!";
        }
    }

    private boolean isGreeting(String message) {  Complexity is 3 Everything is cool!
        return message.contains("Hi") || message.contains("Bye");
    }
}
```

Coverage: All in codecoverage-workshop

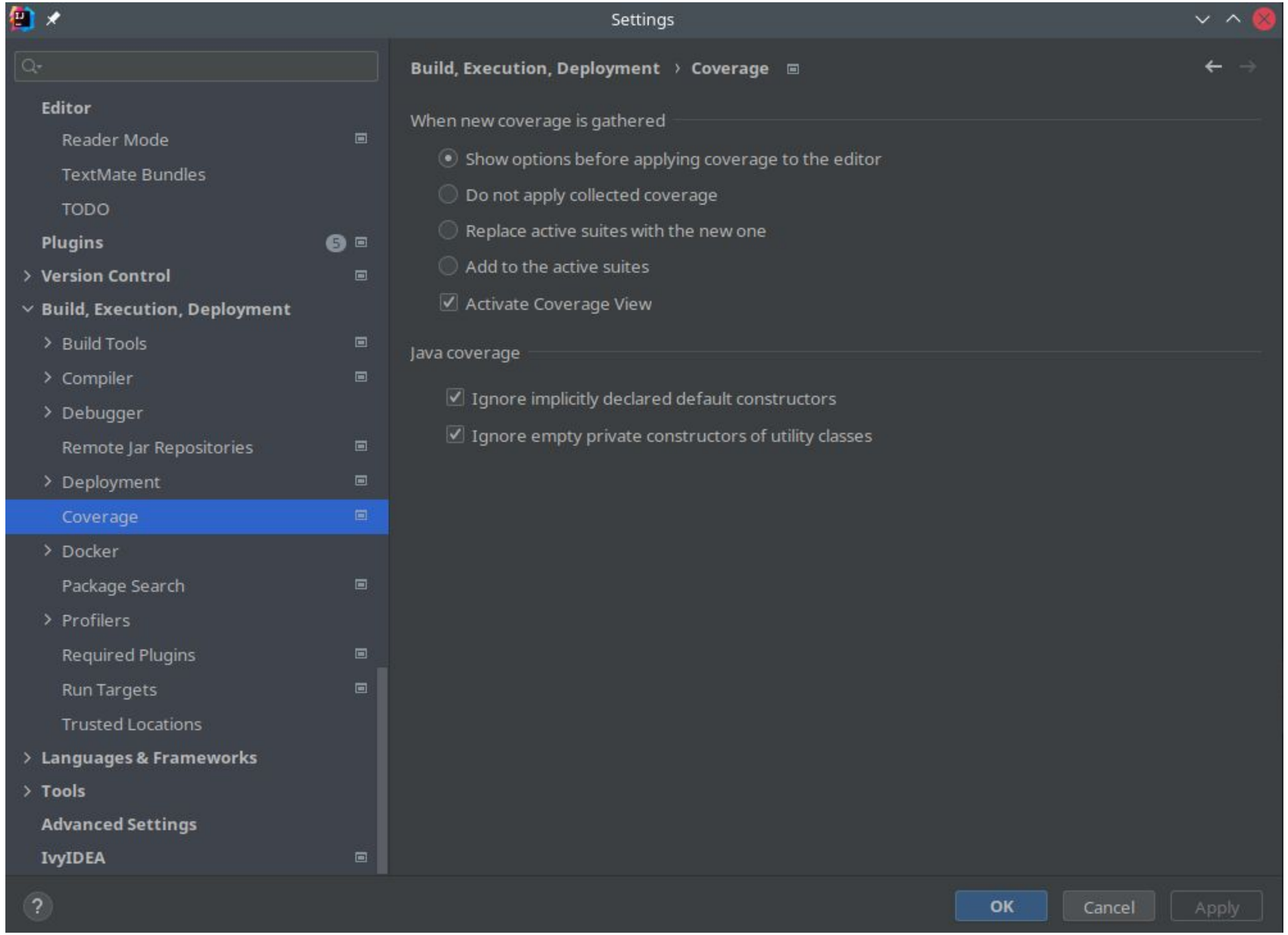
100% classes, 100% lines covered in package 'codecovarage'

Element	Class, %	Method, %	Line, %
ChatBot	100% (1/1)	100% (2/2)	100% (10/10)
FirstChatBot	100% (1/1)	100% (2/2)	100% (3/3)
RefactoredChatBot	100% (1/1)	100% (5/5)	100% (13/13)

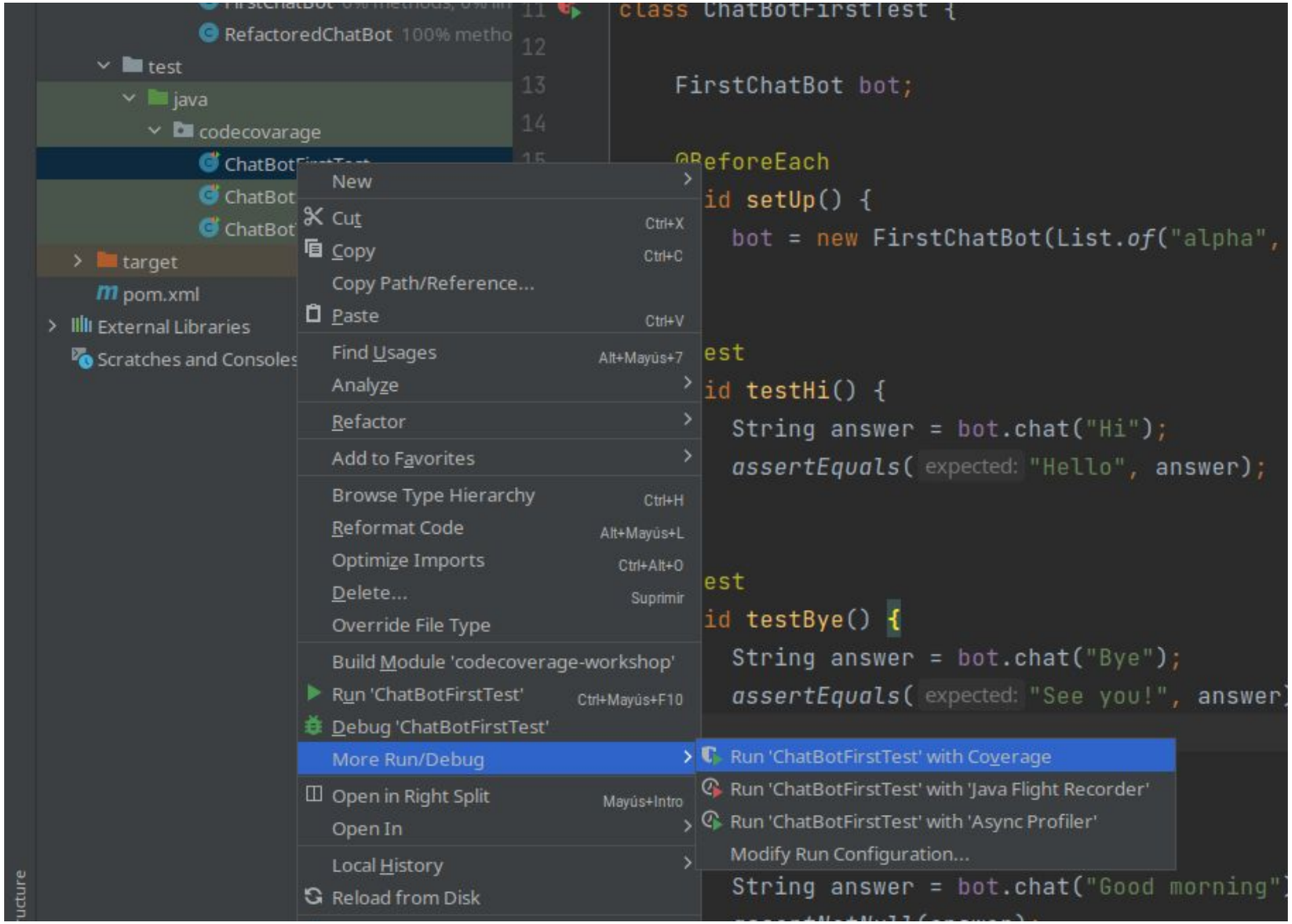
7 Check Code coverage



7 Check Code coverage

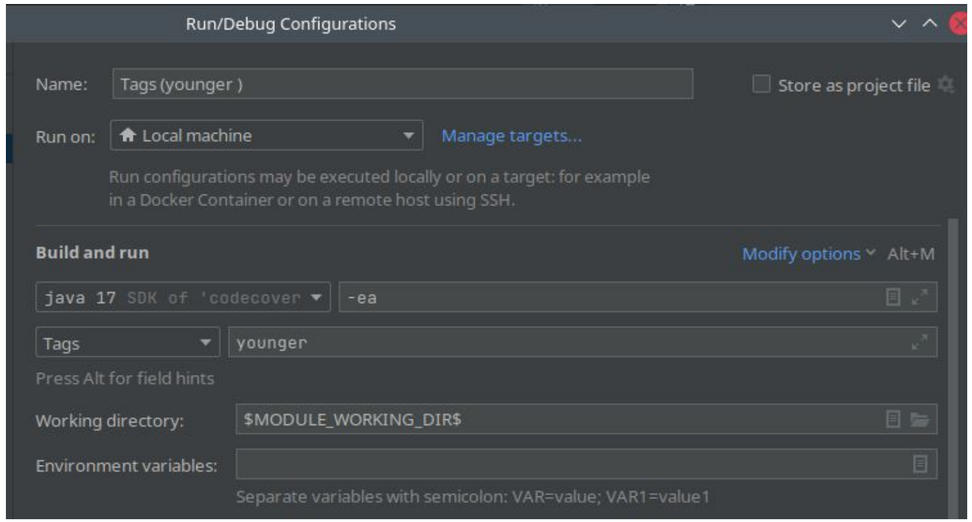


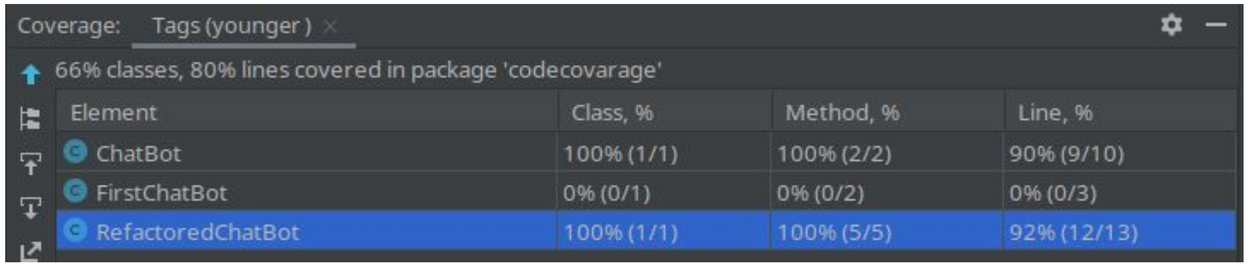
7 Check Code coverage



8 Improve your test

- As we implemented the method using TDD, all your code is covered!
- You can play disabling some tests, so you can see how much code the tests are covering






```
22
23  private String randomMessage(int age) {   Complexity is 5 Everything is cool!
24      if (age > 18) {
25          return randomMessages.get(random.nextInt(randomMessages.size()));
26      } else {
27          return "I'm not allowed to chat with you, sorry.";
28      }
29  }
```

After a first test pass using the Run config above, use this button to run with coverage

Course code

https://github.com/srbarrios/codecoverage-workshop

Code & Test Coverage

Do your tests cover most of the possibilities?

Óscar Barrios

Test Automation Specialist @SUSE